



1. Capture y corra el siguiente programa en el emulador emu8086 y analice el código según los comentarios en el programa, verifique que operaciones si están implementadas.

PAGE 60,80

TITLE PROG3 (EXE) CALCULADORA

; comandos basados una simple calculadora (+[s],[d]).

; ejemplo del calculo:

; input numero1: 10

; input operador: s

; input numero2 : 5

; -----

; 10 + 5 = 5

; salida numero: 5

; -----

; -----

; stack segment

STACKSG segment PARA STACK 'Stack'

DW 64 dup(0)

STACKSG ends

; -----

; data segment

DATASG segment PARA 'Data'

msg0 db "Nota: calculadora solo para valores enteros",0Dh,0Ah

db "ver la salida del resultado de una division float",0Dh,0Ah,'\$'

msg1 db "Ingresa: 1=suma 2=resta 3=mult 4=division 5=salir: \$"

msg2 db 0Dh,0Ah, 0Dh,0Ah, 'Ingresa primer numero: \$'

msg3 db "Ingresa el segundo numero: \$"

msg4 db 0dh,0ah, 'El calculo es : \$'

msg5 db 0dh,0ah,0dh,0ah,'Gracias, Bye! press any key... ', 0Dh,0Ah, '\$'

err_ope db 0Dh,0Ah,"Error operacion no implementada!", 0Dh,0Ah, '\$'

resid1 db 0Dh,0Ah,"Division no exacta ",0Dh,0Ah

db "El residuo en dx es.... \$"

;el operador puede ser: 's','d' o 'q' para salir en medio.

```
opr db '?'
```

```
; primero y segundo numero:
```

```
num1 dw ?
```

```
num2 dw ?
```

```
DATASG ends
```

```
;-----
```

```
;code segment
```

```
CODESG segment PARA 'Code'
```

```
PROG3 PROC FAR
```

```
    ASSUME SS:STACKSG, DS:DATASG, CS:CODESG
```

```
    MOV  AX,DATASG  ;ASIGNA DIRECCION DEL DATASG
```

```
    MOV  DS, AX     ; EN EL REGISTRO DS
```

```
    ; mov dx, offset msg0
```

```
    lea dx, msg0
```

```
    mov ah, 9
```

```
    int 21h
```

```
; nueva linea:
```

```
    putc 0Dh
```

```
    putc 0Ah
```

```
; obtiene operador:
```

```
    lea dx, msg1
```

```
    mov ah, 09h    ; imprime mensaje que esta en ds:dx
```

```
    int 21h
```

```
    mov ah, 1      ; deja solo un char en la entrada en AL.
```

```
    int 21h
```

```
    mov opr, al
```

```
    cmp opr, '2'   ; 2 operacion no implementada.
```

```
    je error_ope
```

```
    cmp opr, '3'   ; 2 operacion no implementada.
```

```
    je error_ope
```

```
    cmp opr, '5'   ; 5 termina programa.
```

```
    je exit
```

```
    cmp opr, '5'   ; 5 termina programa.
```

```

ja error_ope

; obtiene el primer numero
; del teclado, y lo almacena
; en el registro cx:

    lea dx, msg2
    mov ah, 09h    ; imprime mensaje que esta en ds:dx
    int 21h
    call scan_num

; almacena el primer numero:
    mov num1, cx

; nueva linea:
    putc 0Dh
    putc 0Ah

; obtiene el segundo numero
; del teclado, y lo almacena
; en el registro cx:

    lea dx, msg3
    mov ah, 09h    ; imprime mensaje que esta en ds:dx
    int 21h
    call scan_num

; almacena segundo numero:
    mov num2, cx

; calcula
    lea dx, msg4
    mov ah, 09h    ; imprime mensaje que esta en ds:dx
    int 21h

    cmp opr, '1'
    je calcula_suma
    cmp opr, '4'
    je calcula_div

; ninguno de las operaciones de arriba....
error_ope:
    lea dx, err_ope
    mov ah, 09h    ; imprime mensaje que esta en ds:dx

```

int 21h

exit:

```
lea dx, msg5
mov ah, 09h ;imprime mensaje que esta en ds:dx
int 21h
```

; espera que presione cualquier tecla...

```
mov ah, 0
int 16h
```

```
MOV AX, 4c00h ; exit to operating system.
int 21h
```

calcula_suma:

```
mov ax, num1
add ax, num2
call print_num ; imprime valor de ax.
jmp exit
```

calcula_div:

; dx es ignorado (solo trabaja con enteros pequenos).

```
mov dx, 0
mov ax, num1
idiv num2 ; ax = (dx ax) / num2.
push dx
call print_num ; imprime valor de ax.
pop dx
cmp dx, 0
jnz imp_residuo
jmp exit
```

imp_residuo:

```
push dx
lea dx, resid1
mov ah, 09h ; imprime mensaje que esta en ds:dx
int 21h
pop dx
mov ax, dx
call print_num ; imprime valor de ax.
jmp exit
```

PROG3 ENDP ; fin codigo PROG3

ten DW 10 ; used as multiplier/divider by SCAN_NUM & PRINT_NUM_UNNS.

;esta macro imprime el caracter en AL y
;avanza el cursor de la posicion actual

```
PUTC MACRO char
    push  AX
    mov   AL, char
    mov   AH, 0EH
    INT   10H
    pop   AX
ENDM
```

; gets the multi-digit SIGNED number from the keyboard,
; and stores the result in CX register:

```
SCAN_NUM PROC NEAR
    PUSH  DX
    PUSH  AX
    PUSH  SI
    MOV   CX, 0
    ; reset flag:
    MOV   CS:make_minus, 0
```

next_digit:

```
    ; get char from keyboard
    ; into AL:
    MOV   AH, 00h
    INT   16h
    ; and print it:
    MOV   AH, 0Eh
    INT   10h
    ; check for MINUS:
    CMP   AL, '-'
    JE    set_minus
    ; check for ENTER key:
    CMP   AL, 0Dh ; carriage return?
    JNE   not_cr
    JMP   stop_input
```

not_cr:

```
    CMP   AL, 8           ; 'BACKSPACE' pressed?
    JNE   backspace_checked
    MOV   DX, 0           ; remove last digit by
    MOV   AX, CX          ; division:
    DIV   CS:ten          ; AX = DX:AX / 10 (DX-rem).
    MOV   CX, AX
```

```

    PUTC  ' '          ; clear position.
    PUTC  8            ; backspace again.
    JMP  next_digit
backspace_checked:
    ; allow only digits:
    CMP  AL, '0'
    JAE  ok_AE_0
    JMP  remove_not_digit
ok_AE_0:
    CMP  AL, '9'
    JBE  ok_digit
remove_not_digit:
    PUTC  8            ; backspace.
    PUTC  ' '          ; clear last entered not digit.
    PUTC  8            ; backspace again.
    JMP  next_digit ; wait for next input.
ok_digit:
    ; multiply CX by 10 (first time the result is zero)
    PUSH AX
    MOV  AX, CX
    MUL  CS:ten          ; DX:AX = AX*10
    MOV  CX, AX
    POP  AX
    ; check if the number is too big
    ; (result should be 16 bits)
    CMP  DX, 0
    JNE  too_big
    ; convert from ASCII code:
    SUB  AL, 30h
    ; add AL to CX:
    MOV  AH, 0
    MOV  DX, CX          ; backup, in case the result will be too big.
    ADD  CX, AX
    JC   too_big2        ; jump if the number is too big.
    JMP  next_digit

set_minus:
    MOV  CS:make_minus, 1
    JMP  next_digit

too_big2:
    MOV  CX, DX          ; restore the backed up value before add.
    MOV  DX, 0           ; DX was zero before backup!
too_big:

```

```

MOV    AX, CX
DIV    CS:ten ; reverse last DX:AX = AX*10, make AX = DX:AX / 10
MOV    CX, AX
PUTC   8      ; backspace.
PUTC   ' '    ; clear last entered digit.
PUTC   8      ; backspace again.
JMP    next_digit ; wait for Enter/Backspace.

```

```

stop_input:
    ; check flag:
    CMP    CS:make_minus, 0
    JE     not_minus
    NEG    CX
not_minus:
    POP    SI
    POP    AX
    POP    DX
    RET
make_minus    DB    ?      ; used as a flag.
SCAN_NUM     ENDP

```

; este procedimiento imprime un numero en AX,
; usado con PRINT_NUM_UN\$ para imprimir numeros con signo:

```

PRINT_NUM    PROC    NEAR
    PUSH    DX
    PUSH    AX
    CMP     AX, 0
    JNZ     not_zero
    PUTC    '0'
    JMP     printed

```

```

not_zero:
    ; the check SIGN of AX,
    ; make absolute if it's negative:
    CMP     AX, 0
    JNS     positive
    NEG     AX
    PUTC    '-'

```

```

positive:
    CALL    PRINT_NUM_UN$

```

```

printed:

```

```

    POP    AX
    POP    DX
    RET
PRINT_NUM    ENDP

```

; este procedimiento imprime un numero sin signo
; en AX (no solo un digit)
; los valores permitidos van de 0 a 65535 (FFFF)

```

PRINT_NUM_UN$ PROC    NEAR

```

```

    PUSH    AX
    PUSH    BX
    PUSH    CX
    PUSH    DX

```

```

; flag para prevenir imprimir zeros antes que el numero:
MOV    CX, 1
; (resultado de "/ 10000" es siempre menor o igual a 9).
MOV    BX, 10000    ; 2710h - divider.
; AX es zero?
CMP    AX, 0
JZ     print_zero

```

begin_print:

```

; checka el divisor (if zero go to end_print):
CMP    BX,0
JZ     end_print
; evitar imprimir zeros antes del numero:
CMP    CX, 0
JE     calc
; if AX < BX then resultado de DIV sera zero:
CMP    AX, BX
JB     skip

```

calc:

```

MOV    CX, 0    ; set flag.
MOV    DX, 0
DIV    BX    ; AX = DX:AX / BX (DX=remainder).
; imprime el ultimo digito
; AH is always ZERO, so it's ignored

ADD    AL, 30h    ; convert to ASCII code.
PUTC    AL
MOV    AX, DX    ; get remainder from last div.

```

skip:

```
; calculate BX=BX/10
PUSH  AX
MOV   DX, 0
MOV   AX, BX
DIV   CS:ten ; AX = DX:AX / 10 (DX=remainder).
MOV   BX, AX
POP   AX
JMP   begin_print
```

print_zero:

```
PUTC  '0'
```

end_print:

```
POP   DX
POP   CX
POP   BX
POP   AX
RET
```

PRINT_NUM_UNS ENDP

CODESG ENDS

END PROG3

- 2. Agregue su nombre y secuencia para que se impriman a la salida, implemente las operaciones que faltan.**

- 3. Adicionar y Enviar en el Archivo de Word con su nombre de la práctica anterior, los resultados obtenidos de las operaciones que pudo realizar en la SALIDA de la corrida (Pantallas).**

```
emulator screen (80x25 chars)
SANCHEZ OLVERA JOSE ELIAS SECUENCIA:1CV29
Nota: calculadora solo para valores enteros
ver la salida del resultado de una division float

Ingresa: 1=suma 2=resta 3=mult 4=division 5=salir: 1
Ingresa primer numero: 4
Ingresa el segundo numero: 44
El calculo es: 48
Gracias, Bye! press any key...
```

```
emulator screen (80x25 chars)
SANCHEZ OLVERA JOSE ELIAS SECUENCIA:1CV29
Nota: calculadora solo para valores enteros
ver la salida del resultado de una division float

Ingresa: 1=suma 2=resta 3=mult 4=division 5=salir: 2
Ingresa primer numero: 40
Ingresa el segundo numero: 23
El calculo es: 17
Gracias, Bye! press any key...
```

```
emulator screen (80x25 chars)
SANCHEZ OLVERA JOSE ELIAS SECUENCIA:1CV29
Nota: calculadora solo para valores enteros
ver la salida del resultado de una division float

Ingresa: 1=suma 2=resta 3=mult 4=division 5=salir: 3

Ingresa primer numero: 10
Ingresa el segundo numero: 80
El calculo es: 800

Gracias, Bye! press any key...

clear screen  change font  0/16
```

```
emulator screen (80x25 chars)
SANCHEZ OLVERA JOSE ELIAS SECUENCIA:1CV29
Nota: calculadora solo para valores enteros
ver la salida del resultado de una division float

Ingresa: 1=suma 2=resta 3=mult 4=division 5=salir: 4

Ingresa primer numero: 80
Ingresa el segundo numero: 14
El calculo es: 5
Division no exacta ....
El residuo en dx es.... 10

Gracias, Bye! press any key...

clear screen  change font  0/16
```

```
emulator screen (80x25 chars)
SANCHEZ OLVERA JOSE ELIAS SECUENCIA:1CV29
Nota: calculadora solo para valores enteros
ver la salida del resultado de una division float

Ingresa: 1=suma 2=resta 3=mult 4=division 5=salir: 4

Ingresa primer numero: 10
Ingresa el segundo numero: 2
El calculo es: 5

Gracias, Bye! press any key...

clear screen  change font  0/16
```

